Thesis-1.pdf

Indian Institute of Technology Jammu

Document Details

Submission ID
trn:oid:::3117:586406650

Submission Date
May 5, 2026, 6:52 PM GMT+5:30

Download Date
May 5, 2026, 7:22 PM GMT+5:30

File Name
Thesis-1.pdf

File Size
2.9 MB

70 Pages

13,172 Words

70,360 Characters

35% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

67 AI-generated only 35%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products

DEEPAK GUPTA
2024PCS0024

A Dissertation Submitted to

Indian Institute of Technology, Jammu

In Partial Fulfillment of the Requirements for the Degree of

Master of Technology

in
Computer Science and Engineering

भारतीय प्रौद्योगिकी संस्थान जम्मू
INDIAN INSTITUTE OF TECHNOLOGY JAMMU

Department of Computer Science & Engineering

May 2026

DECLARATION

This thesis is submitted as a requirement for the award of the Master of Technology degree in Computer Science and Engineering in the Department of Computer Science and Engineering. I declare that this written submission represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. The research embodied in this thesis has not been submitted in part or full, to any other University or Institute for the award of any other degree or diploma before. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources that have thus not been properly cited, or from whom proper permission has not been taken when needed.

Name: Deepak Gupta
Roll No.: 2024PCS0024

APPROVAL SHEET

This thesis titled **Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products** by **Deepak Gupta** is approved for the degree of **Master of Technology in Computer Science and Engineering** from IIT Jammu.

Date:

Signature:_____

Name, Institute
External Examiner

Signature:_____

Name, IIT Jammu
Internal Examiner

Signature:_____

Name, IIT Jammu
Chairman

Signature:_____

Dr. Yamuna Prasad, IIT Jammu
Supervisor

To my parents,
who gave me the courage to dream big,
and to my teachers,
who showed me how to turn those dreams into reality.

Acknowledgement

This thesis is the product of two years of work that I could not have done alone, and I want to thank the people who made it possible.

My deepest and most heartfelt gratitude is owed to my supervisor, **Dr. Yamuna Prasad**, Head of the Department, Department of Computer Science and Engineering, IIT Jammu, whose mentorship has been the formative experience of my time as a graduate student. I am thankful for his invaluable guidance, his unwavering encouragement, and the trust and freedom he gave me to pursue ideas across privacy-preserving federated learning, applied cryptography, and adversarial machine-learning security. His insights at critical junctures shaped both the direction and the rigour of this work, and the clarity of thought he demands has been the most enduring lesson of my time as his student.

I am thankful to my fellow M.Tech and Ph.D. scholars at IIT Jammu for the many late-evening conversations, technical discussions, and shared moments of encouragement that helped this work take shape. Those exchanges made the long research process more thoughtful, collaborative, and memorable.

I am thankful to the Department of Computer Science and Engineering at IIT Jammu for the compute resources, the coursework, and the academic environment that made this research possible, and to the faculty members who taught me during my first year.

Finally, I extend my heartfelt gratitude to my parents and my family for their constant support and encouragement throughout the course of this work.

Deepak Gupta

List of Publications

Conference Publications

1. Mahendra Kumar Gurve, Deepak Gupta, Nitin, and Yamuna Prasad, "Eliminating the Auditor: Functional Inner Product Based Byzantine-Resilient Federated Learning," accepted as a poster at the *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026)*, Saint-Malo, France, May 2026.

iThenticate Page 13 of 72 - AI Writing Submission Submission ID trn:oid:::3117:586406650

Abstract

Federated Learning (FL) lets many clients train a shared model without exposing their raw data. To strengthen the privacy guarantee, modern deployments wrap each client's update in additive homomorphic encryption, giving rise to Privacy-Preserving Federated Learning (PPFL). The same encryption that hides gradients from a curious server, however, also hides whether they are honest: a Byzantine client can flip labels before training and quietly poison the aggregated model, either degrading overall accuracy or installing a backdoor on a chosen target class.

Existing defences in this setting rely on auditor-based architectures that insert a trusted third party to decrypt cluster-level statistics of the incoming updates and screen Byzantine clients with distributional models. While effective, such schemes reintroduce the very trust anchor PPFL was designed to remove, expose coarse structural information about the encrypted gradients, and scale super-linearly in the model dimension. A defence that is simultaneously fully encrypted, free of any new trust anchor, and linear in the model dimension has remained an open challenge.

This thesis proposes **FIP-FL**, an auditor-free Byzantine-resilient PPFL framework. The central idea is to verify each client's encrypted gradient through a single scalar — its *Functional Inner Product* (FIP) with a momentum-tracked trusted reference direction — and to filter the resulting scores using a two-stage geometric audit: a direction filter followed by a robust magnitude window. I establish three formal guarantees of the scheme: gradient privacy, attack resilience, and monotone loss decrease under an L-smooth objective.

The framework is implemented end-to-end and evaluated on MNIST and KDDCup99, under both IID and non-IID client partitions, and against targeted and untargeted label-flipping attacks at malicious-client rates up to 50%. Across this evaluation grid, FIP-FL achieves strong robustness without any external auditor. The research underlying this thesis has been accepted as a poster at the ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026), Saint-Malo, France, May 2026.

Contents

List of Figures

List of Tables

List of Abbreviations

ACC — Overall Accuracy

BFV — Brakerski–Fan–Vercauteren (homomorphic encryption scheme)

BGV — Brakerski–Gentry–Vaikuntanathan (homomorphic encryption scheme)

CKKS — Cheon–Kim–Kim–Song (approximate homomorphic encryption)

DCR — Decisional Composite Residuosity

DoS — Denial of Service

DP — Differential Privacy

FedAvg — Federated Averaging

FHE — Fully Homomorphic Encryption

FIP — Functional Inner Product

FIP-FL — Functional Inner Product Federated Learning (proposed framework)

FL — Federated Learning

FPR — False Positive Rate

GMM — Gaussian Mixture Model

HE — Homomorphic Encryption

IID — Independent and Identically Distributed

KDD — Knowledge Discovery in Databases

MAD — Median Absolute Deviation

MNIST — Modified National Institute of Standards and Technology (database)

$O_{\mathbf{acc}}$ — Other-label Accuracy

PCA — Principal Component Analysis

PPFL — Privacy-Preserving Federated Learning

R2L — Remote-to-Local (KDDCup attack class)

SenSys — ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems

SGD — Stochastic Gradient Descent

SHA — Secure Hash Algorithm

SMPC — Secure Multiparty Computation

$T_{\mathbf{acc}}$ — Target-class Accuracy

TF-IDF — Term Frequency – Inverse Document Frequency

TKA — Trusted Key Authority

TPR — True Positive Rate

U2R — User-to-Root (KDDCup attack class)

1 Introduction

1.1 Background and Context

Federated Learning (FL), introduced by McMahan et al. [2], trains a global model without moving raw data into a central repository. Each client keeps its local samples private and sends only a gradient or parameter update, making FL useful for mobile keyboards, medical data, fraud detection, and other sensitive distributed settings.

Keeping data local is not sufficient by itself: shared gradients can leak training samples [3], membership, or sensitive distributional attributes. Privacy-Preserving Federated Learning (PPFL) protects updates through differential privacy, secure aggregation, or homomorphic encryption. This thesis focuses on additive homomorphic encryption, where the server can aggregate encrypted updates without inspecting individual gradients.

1.2 The Research Gap

Encryption protects privacy, but it also hides the evidence needed for Byzantine defence. A malicious client can flip labels, scale its update, or submit a crafted poison vector while the server sees only ciphertexts. Robust aggregation rules such as Krum [8], coordinate-wise median and trimmed-mean [9], and Bulyan [10] assume plaintext updates; detection-based defences such as Auror [11], FL-Trust [12], and PEFL [13] similarly require plaintext gradients or revealing summaries.

The closest encrypted-gradient baseline is the auditor-based scheme of Yazdinejad et al. [1]. It is effective, but relies on a trusted auditor, decrypts

cluster-level summaries, incurs an $\mathcal{O}(d^3)$ covariance-inversion cost, and exposes population structure under non-IID data. The missing piece is an encrypted-gradient defence that is auditor-free, robust, linear in model dimension, and usable under realistic non-IID partitions.

1.3 Research Objectives and Contributions

This thesis designs, analyses, and evaluates FIP-FL, an auditor-free Byzantine-resilient PPFL framework. FIP-FL verifies encrypted client updates through a functional inner product (FIP) with a trusted reference direction, reveals only one scalar score per client per round, and applies a two-stage geometric audit: a direction filter followed by a median-and-MAD magnitude window.

The main contributions are:

1. **An auditor-free PPFL architecture.** FIP-FL removes the external auditor and performs verification from encrypted updates using a scalar FIP score.
2. **A two-stage geometric audit.** The direction filter rejects misaligned updates, while the magnitude window rejects weak or over-scaled positive updates before aggregation.
3. **Formal guarantees.** The thesis proves scalar-gradient privacy, attack admissibility, monotone descent under L-smoothness, and lower verifier complexity than the auditor-based baseline.
4. **Empirical validation.** FIP-FL is evaluated on MNIST and KDDCup99 under IID and non-IID partitions, targeted and untargeted label flipping, and malicious-client rates up to 50%.
5. **Engineering efficiency.** The verifier runs in $\mathcal{O}(Nd + N \log N)$ rather than using an $\mathcal{O}(d^3)$ covariance step, and Phase II parallelisation gives a 13× speed-up on 25 cores.

The research underlying this thesis has been accepted as a poster at the ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026), Saint-Malo, France, May 2026.

2 Background and Motivation

2.1 Federated Learning and FedAvg

Consider N clients, where client i holds private data $\mathcal{D}_i = \{(x_k^{(i)}, y_k^{(i)})\}_{k=1}^{n_i}$ and $n = \sum_i n_i$. The federated objective is the weighted empirical risk

$$F(\mathbf{w}) = \sum_{i=1}^{N} \frac{n_i}{n} F_i(\mathbf{w}), \qquad F_i(\mathbf{w}) = \frac{1}{n_i} \sum_{k=1}^{n_i} \ell(f_{\mathbf{w}}(x_k^{(i)}), y_k^{(i)}). \tag{2.1}$$

FedAvg [2] optimises this objective by broadcasting $\mathbf{w}^{(t)}$, running local SGD at each client, and aggregating returned updates. With descent-direction update $\Delta\boldsymbol{\theta}_i^{(t)} = \mathbf{w}^{(t)} - \mathbf{w}_i^{(t+1)}$, the server update is

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \sum_{i=1}^{N} \frac{n_i}{n} \Delta\boldsymbol{\theta}_i^{(t)}. \tag{2.2}$$

The server therefore needs only coordinate-wise aggregation, while raw samples remain local.

2.2 IID and Non-IID Federations

FedAvg is simplest when client data are IID draws from a common population. Real federations are usually non-IID: users, hospitals, and network sites observe different label distributions and feature patterns. This thesis uses label skew, where each client receives a subset of labels with controlled overlap. Label skew makes local objectives F_i drift from the global objective F, so honest gradients can differ substantially. A Byzantine defence must therefore reject malicious deviation without treating legitimate heterogeneity as an attack.

2.3 Privacy-Preserving Federated Learning

Gradients can reveal training data through reconstruction, membership inference, or property inference attacks [3]. PPFL protects the update channel using three main mechanisms. Differential privacy clips updates and adds calibrated noise, but can reduce accuracy and resemble adversarial perturbation. Secure aggregation [14] hides individual updates but usually reveals only the aggregate, which is too late for per-client filtering. Homomorphic encryption lets the server compute on ciphertexts; Paillier [4] supports encrypted addition and plaintext-scalar multiplication, while BFV/CKKS [5] support richer packed computation at higher cost.

FIP-FL builds on Paillier because its verification step is linear: each encrypted update is scored by an inner product against a plaintext reference direction. This avoids DP noise and peer-to-peer client protocols while keeping encrypted computation linear in the model dimension.

2.4 Poisoning Attack Taxonomy

A Byzantine client may send any syntactically valid update. This thesis evaluates two label-flipping attacks. In an *untargeted* attack, a malicious client remaps labels by a fixed cyclic offset and tries to reduce overall accuracy. In a *targeted* attack, it maps one source class to one target class, aiming to damage a specific inference case while leaving the rest of the task plausible. Other Byzantine attacks include sign flipping, scaling, model replacement, and collusion; these motivate the direction and magnitude components of the FIP-FL audit, even though the empirical grid focuses on label flipping.

3 Literature Survey

3.1 Robust Aggregation Rules

Robust aggregation replaces the FedAvg mean with an estimator that can tolerate adversarial updates. Krum and Multi-Krum [8] select updates close to many neighbours, but require plaintext pairwise distances and cost $\mathcal{O}(N^2 d)$. Coordinate-wise median and trimmed-mean [9] are cheaper at $\mathcal{O}(Nd \log N)$, but their IID-style assumptions weaken under label skew, where honest coordinate distributions can be multi-modal. Bulyan [10] combines Krum-style selection with trimmed averaging, improving robustness while retaining plaintext access and client-count constraints.

These methods are important baselines, but they do not transfer directly to encrypted PPFL because the server cannot compute distances, medians, or coordinate-wise trims over hidden updates without decryption.

3.2 Detection-Based Defences

Detection-based defences identify malicious clients and remove them before aggregation. Auror [11] clusters update features and flags minority clusters; it is fragile when non-IID honest clients naturally form several clusters. FL-Trust [12] scores plaintext updates against a server root gradient and is conceptually close to FIP-FL, but it assumes access to plaintext updates. PEFL [13] moves correlation scoring into the encrypted setting, yet reported robustness is weak under high attack rates and non-IID partitions [1]. FoolsGold [15] detects colluding clients through historical similarity, while ShieldFL [16] uses a helper-assisted secure protocol for encrypted-gradient detection.

The common pattern is that stronger detection usually costs either plaintext access, extra trusted parties, or additional communication. FIP-FL targets the middle ground: encrypted individual updates, no auditor, and one scalar score per client.

3.3 Auditor-Based PPFL Baseline

Yazdinejad et al. [1] provide the central baseline for this thesis. Their system inserts a trusted auditor between the clients and the aggregation server. The auditor decrypts cluster-level summaries, fits a Gaussian mixture model in a low-dimensional projection, and applies Mahalanobis-distance outlier screening before forwarding trusted updates for homomorphic aggregation.

The baseline reports strong accuracy, including about 96.5% target-class accuracy on MNIST non-IID with a 50% targeted label-flipping attack. Its limitations are structural rather than merely empirical: the auditor is an additional trust anchor, the covariance inversion is $\mathcal{O}(d^3)$, and decrypted cluster statistics leak population structure. These are precisely the limitations addressed by FIP-FL.

3.4 Positioning of FIP-FL

FIP-FL differs from the above work in four ways. It keeps individual updates encrypted, introduces no external auditor or helper, replaces distributional clustering with geometric alignment to a trusted reference direction, and keeps verifier work linear in d. Compared with plaintext robust aggregation, it preserves PPFL privacy; compared with auditor-based PPFL, it removes decrypted cluster statistics and the cubic Mahalanobis step; compared with root-gradient methods, it computes the alignment score homomorphically. The next chapter formalises the primitives needed to build this design.

4 Preliminaries

4.1 Notation

Scalars are lowercase italics (a, η), vectors are lowercase bold ($\mathbf{x}$, $\boldsymbol{\theta}$), and matrices are uppercase bold ($\mathbf{W}$, $\mathbf{M}$). For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$,

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{k=1}^{d} a_k b_k, \qquad \|\mathbf{a}\|_2 = \sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}. \tag{4.1}$$

A Paillier ciphertext is written $[\![m]\!] = \mathrm{Enc}_{pk}(m)$. The main FL symbols are N clients, model dimension d, round index t, local epochs E, batch size B, learning rate η, and Paillier security parameter λ.

4.2 The Paillier Cryptosystem

Paillier [4] is a probabilistic additively homomorphic public-key scheme. Given security parameter λ, the Trusted Key Authority samples primes p, q, sets $n = pq$, uses generator $g = n + 1$, and keeps the secret key $sk = (\ell, \alpha)$ where $\ell = \mathrm{lcm}(p - 1, q - 1)$ and $\alpha = \ell^{-1} \bmod n$. This thesis uses $\lambda = 1024$.

For plaintext $m \in \mathbb{Z}_n$ and random $r \in \mathbb{Z}_n^*$,

$$\mathrm{Enc}_{pk}(m) = g^m r^n \pmod{n^2}, \tag{4.2}$$

and decryption is

$$\mathrm{Dec}_{sk}(c) = L(c^\ell \bmod n^2)\alpha \pmod{n}, \qquad L(x) = \frac{x - 1}{n}. \tag{4.3}$$

For plaintexts m_1, m_2 and known integer a,

$$\mathrm{Enc}_{pk}(m_1)\mathrm{Enc}_{pk}(m_2) \equiv \mathrm{Enc}_{pk}(m_1 + m_2) \pmod{n^2}, \tag{4.4}$$

$$\mathrm{Enc}_{pk}(m_1)^a \equiv \mathrm{Enc}_{pk}(am_1) \pmod{n^2}. \tag{4.5}$$

These identities are sufficient for encrypted sums and linear functionals; FIP-FL never needs multiplication between two encrypted values.

Real gradients are encoded as fixed-point integers. For scale s,

$$\mathrm{encode}(x) = \mathrm{round}(sx), \qquad \mathrm{decode}(z) = z/s. \tag{4.6}$$

Signed integers are represented in the Paillier plaintext ring by centered modular encoding: a negative value $-a$ is stored as $n - a$ and decoded back from the interval $(-n/2, n/2]$. Negative plaintext scalars in homomorphic multiplication are implemented through modular inverses. The experiments use $s = 10{,}000$. Since an encrypted FIP score multiplies an encoded update coordinate by an encoded reference coordinate, the decrypted score is divided by s^2.

4.3 Functional Inner Product

For gradient vectors, the Functional Inner Product used in this thesis is the Euclidean dot product

$$\langle \mathbf{g}_i, \mathbf{g}_{\mathrm{ref}} \rangle = \sum_{k=1}^{d} g_i^{(k)} g_{\mathrm{ref}}^{(k)}. \tag{4.7}$$

It measures whether a client update aligns with the trusted reference direction. Combining (4.7) with Paillier scalar multiplication and addition gives the homomorphic score

$$[\![\langle \mathbf{g}_i, \mathbf{g}_{\mathrm{ref}} \rangle]\!] = \prod_{k=1}^{d} [\![g_i^{(k)}]\!]^{\mathrm{encode}(g_{\mathrm{ref}}^{(k)})} \pmod{n^2}. \tag{4.8}$$

The TKA decrypts only this scalar. The server never observes a coordinate of $\mathbf{g}_i$.

4.4 Robust Statistics

FIP-FL thresholds one scalar score per client, so one-dimensional robust statistics are enough. For score set $\mathcal{S} = \{s_1, \ldots, s_N\}$,

$$\tilde{\mu} = \text{median}(\mathcal{S}), \qquad \text{MAD} = \text{median}\{|s_i - \tilde{\mu}| : s_i \in \mathcal{S}\}. \tag{4.9}$$

The median and MAD remain stable under contamination near the 50% boundary. FIP-FL uses the adaptive window

$$\tau_{\min}^{(t)} = \alpha(\rho^{(t)})\tilde{\mu}^{(t)}, \qquad \tau_{\max}^{(t)} = \tilde{\mu}^{(t)} + \gamma_{\text{mad}}\text{MAD}^{(t)}, \tag{4.10}$$

with $\gamma_{\text{mad}} = 5$. Scores below the window behave like weak positive contributions, while scores above it indicate an aligned update with suspiciously large influence.

4.5 System and Threat Model

FIP-FL has four roles: clients submit encrypted gradients; the Aggregation Server combines accepted ciphertexts and broadcasts the model; the FIP Verifier computes scalar FIP scores and applies the two-stage audit; and the Trusted Key Authority generates Paillier keys and decrypts only protocol-specified scalar scores or aggregate updates. The Verifier may be co-located with the server; no external auditor is introduced.

The adversary controls up to $f < N/2$ Byzantine clients. It may relabel local data, submit arbitrary valid ciphertexts, collude across malicious clients, and observe public broadcasts. The experiments instantiate this model with targeted and untargeted label flipping at malicious rates $f/N \in \{10\%, 20\%, 30\%, 50\%\}$.

4.6 Security and Privacy Goals

The protocol has four goals. *Privacy*: the server and Verifier learn no client-gradient coordinate, norm, or pairwise distance beyond the scalar score $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{g}_{\text{ref}}^{(t)} \rangle$. *Byzantine resilience*: accepted updates must have positive direction and score magnitude inside $(\tau_{\min}^{(t)}, \tau_{\max}^{(t)})$. *Convergence*: if the accepted aggregate

has positive projection on the reference, then an L-smooth objective decreases for step size $\eta \leq 1/L$. *Efficiency*: verifier work is $\mathcal{O}(Nd + N \log N)$ and aggregation work is $\mathcal{O}(|\mathcal{A}^{(t)}|d)$ for accepted set $\mathcal{A}^{(t)}$.

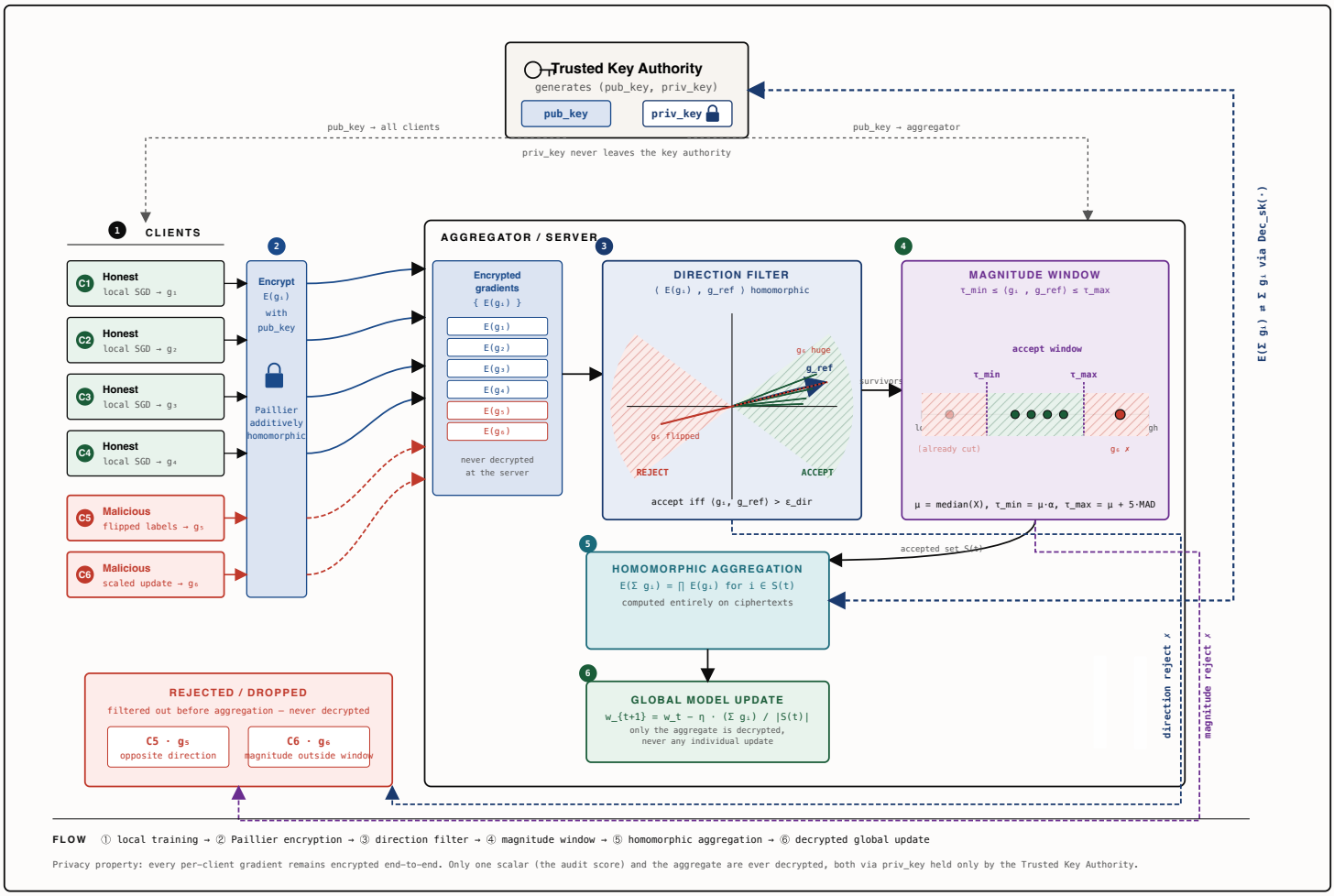
5 Proposed Framework: FIP-FL

5.1 System Architecture

FIP-FL has four logical entities: N clients, an Aggregation Server, a FIP Verifier, and a Trusted Key Authority (TKA). Figure 5.1 shows the single-round privacy-preserving pipeline: clients expose Paillier ciphertexts, the Verifier derives one alignment score per client, the server receives only accepted ciphertexts, and the TKA decrypts only scalar scores or aggregate updates.

FIP-PPFL — Functional Inner Product · Privacy-Preserving Federated Learning

A trusted key authority publishes a Paillier public key. Clients train locally and submit encrypted gradients. The aggregator audits each encrypted update against a benign reference using two geometric criteria — first a directional check, then a magnitude window — and homomorphically aggregates only the survivors. The single private key never leaves the key authority.



① TRUSTED KEY AUTHORITY

Generates one Paillier keypair. Distributes the public key to all clients and the aggregator; keeps the private key sealed and uses it only for decryption of the audit score and the aggregate.

```
pub_key → broadcast
priv_key → never leaves
```

③ DIRECTION FILTER

Server computes the homomorphic inner product $\langle g_i, g_{ref} \rangle$ against a benign reference built on public data. Updates pointing in the wrong half-space are removed before any aggregation.

$$\text{accept iff } \langle g_i, g_{ref} \rangle > \varepsilon_{dir}$$

④ MAGNITUDE WINDOW

Robust statistics on the alignment scores. Median μ and MAD define an adaptive window; any client whose score is too small or too large is dropped — symmetric protection against shrinking and scaling attacks.

$$\tau_{min} \le \langle g_i, g_{ref} \rangle \le \tau_{max}$$

⑤ SECURE AGGREGATION

The encrypted gradients of the surviving set S(t) are summed homomorphically. Only the aggregate is sent to the key authority for decryption — individual gradients never appear in plaintext.

$$E(\Sigma\, g_i) = \prod E(g_i),\ i \in S(t)$$

FIP-PPFL · System flow · Paillier additively homomorphic encryption

Geometric audit · direction + magnitude · no plaintext leak

The separation keeps responsibilities narrow: clients train and encrypt, the Verifier scores and filters, the server aggregates accepted ciphertexts, and the TKA performs controlled decryptions. The Verifier sees encrypted updates and the plaintext reference $\mathbf{R}^{(t)}$, but never an individual plaintext gradient; the TKA decrypts scalar FIP scores and aggregate updates, but does not run the audit.

5.2 System Setup and Key Distribution

Setup runs once before round 1. Algorithm 1 separates cryptographic state, model state, and geometric state: the TKA creates the encryption domain, the server starts from a clean global model, and the Verifier stores a unit-norm reference direction from public bootstrap data. Later rounds refresh this direction from accepted aggregate updates.

FIGURE · ALGORITHM 1

FIP-PPFL System Initialization and Reference Bootstrapping

A geometric view of the three setup phases — key generation & registration, model initialization, and reference bootstrapping — culminating in a unit-norm trusted reference direction $g_{ref}^{(0)} \in \mathbb{R}^d$ stored at the server-side FIP Verifier module for round 1.

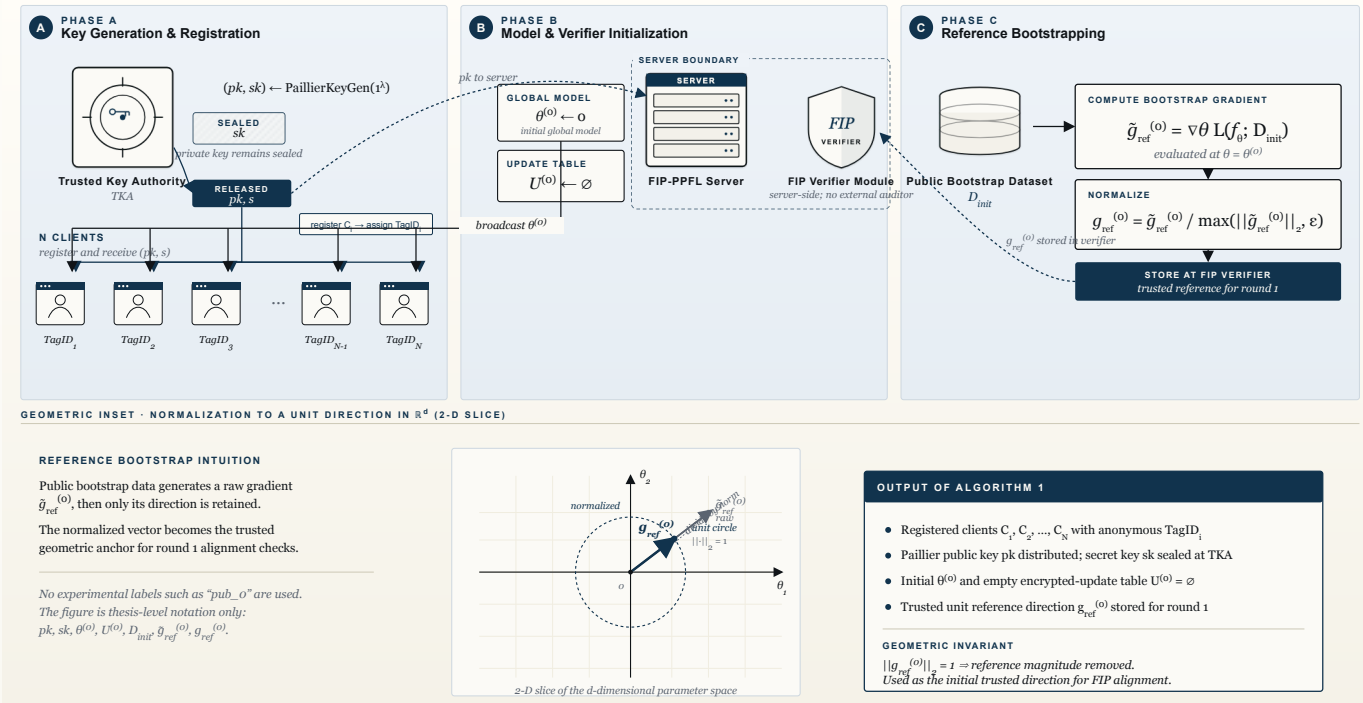


Figure 1. Algorithm 1. The TKA generates the Paillier key pair and keeps *sk* sealed while releasing only *pk* and the fixed-point scale *S*. The server initializes $\theta^{(0)} \leftarrow \text{InitModel}(f_\theta)$ (zero initialization for the logistic-regression experiments) together with $U^{(0)} = \varnothing$, broadcasts $\theta^{(0)}$ to the registered clients, and the server-side FIP Verifier stores the initial trusted reference direction. The trusted public bootstrap set D_{init} produces the raw gradient $\tilde{g}_{\text{ref}}^{(0)}$, which is normalized to $g_{\text{ref}}^{(0)}$ using $\max(\|\tilde{g}_{\text{ref}}^{(0)}\|_2, \varepsilon)$. The bottom inset shows the geometric normalization from a raw gradient vector to a unit direction in a 2-D slice of the parameter

Algorithm 1 FIP-PPFL System Initialization and Reference Bootstrapping

Require: Number of clients N, model $f_{\boldsymbol{\theta}}$, public bootstrap set $\mathcal{D}_{\text{init}}$, security parameter λ, fixed-point scale s

Ensure: Registered clients, Paillier keys, initial global model $\boldsymbol{\theta}^{(0)}$, trusted reference direction $\mathbf{g}_{\text{ref}}^{(0)}$

Key generation and registration

$(pk, sk) \leftarrow \textsc{PaillierKeyGen}(1^{\lambda})$ at the Trusted Key Authority

TKA keeps sk sealed and releases only pk to the clients and the FIP-PPFL server

for $i \leftarrow 1$ **to** N **do**

    Register client C_i and assign anonymous tag TagID_i

    Send (pk, s) to client C_i

end for

Model and verifier initialisation

Initialise global model parameters $\boldsymbol{\theta}^{(0)} \leftarrow \mathbf{0}$

Broadcast $\boldsymbol{\theta}^{(0)}$ to all registered clients

Initialise encrypted-update table $\mathcal{U}^{(0)} \leftarrow \emptyset$

Reference bootstrapping

Compute the trusted bootstrap gradient on public data:

$\widetilde{\mathbf{g}}_{\text{ref}}^{(0)} \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}; \mathcal{D}_{\text{init}}) \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(0)}}$

Normalise the reference direction:

$\mathbf{g}_{\text{ref}}^{(0)} \leftarrow \widetilde{\mathbf{g}}_{\text{ref}}^{(0)} / \max\left(\left\| \widetilde{\mathbf{g}}_{\text{ref}}^{(0)} \right\|_2, \varepsilon \right)$

Store $\mathbf{g}_{\text{ref}}^{(0)}$ at the FIP Verifier for round 1

The bootstrap uses a small public, non-sensitive task-domain subset to compute $\mathbf{g}_{\text{ref}}^{(0)}$ at the zero model. In later rounds the same reference is written as $\mathbf{R}^{(t)}$.

5.3 Per-Round Protocol

Each round $t \geq 1$ follows the poster flow: encrypted gradient, FIP score, direction filter, magnitude window, secure aggregation, and reference update.

5.3.1 Phase I: Local Training and Encrypted Update Generation

Each client trains for E local epochs, averages its mini-batch gradients, flattens weights and bias into one d-dimensional update, encrypts each coordinate under Paillier, and sends the ciphertexts to the Verifier. Algorithm 2 gives the client-side procedure; malicious clients may first apply label flipping.

FIGURE · ALGORITHM 2

Client-Side Local Training and Encrypted Gradient Submission

A geometric view of the client-side round procedure — receiving the round model, performing local mini-batch training, averaging and flattening the local gradient, encoding each coordinate with fixed-point scale S, encrypting under Paillier public key pk, and sending the anonymous encrypted submission $M_i^{(t)}$ to the FIP Verifier.

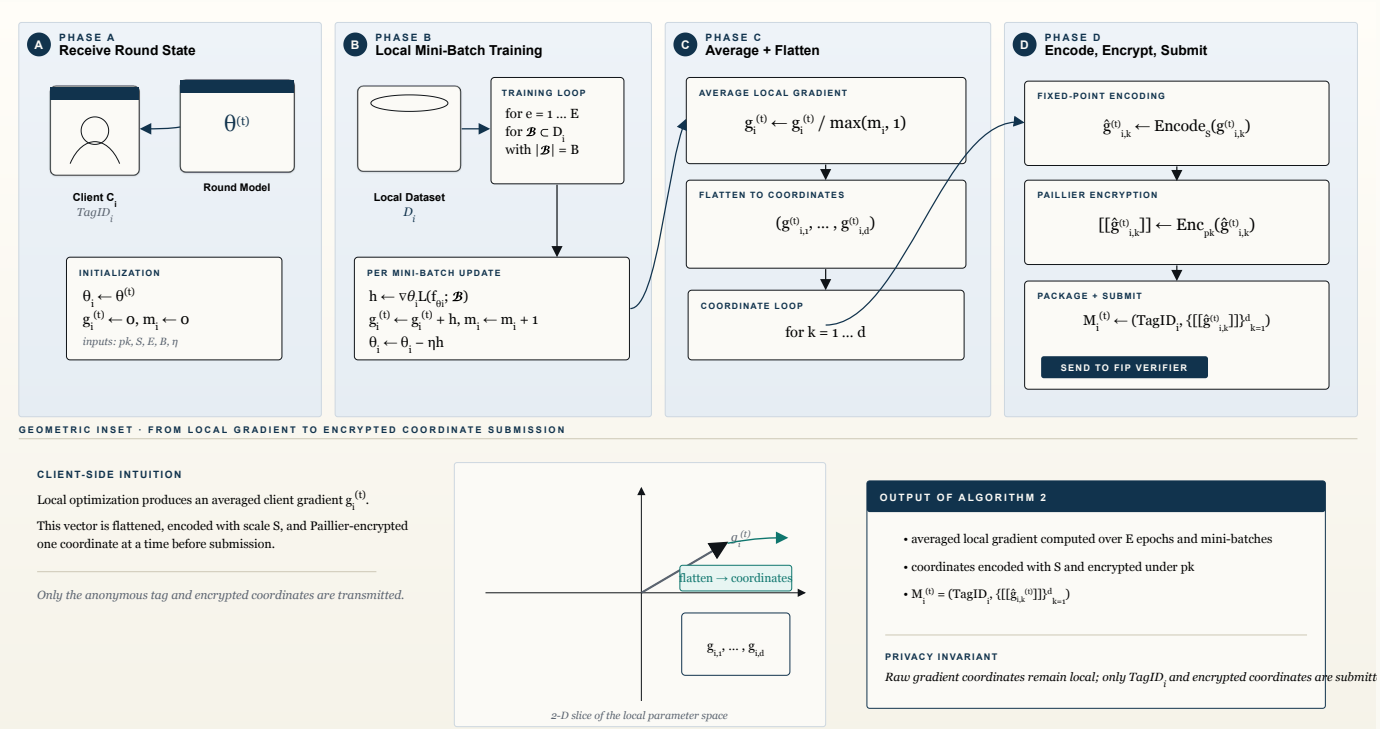


Figure 2. Algorithm 2. Client C_i receives the round model $\theta^{(t)}$, public key *pk*, fixed-point scale *S*, and local hyperparameters. It initializes $\theta_i \leftarrow \theta^{(t)}$, performs local mini-batch SGD over *E* epochs, accumulates and averages the local gradient $g_i^{(t)}$, and then flattens it into coordinates. Each coordinate is encoded as $\hat{g}_{i,k}{}^{(t)}$ using *S* and encrypted as $[[\hat{g}_{i,k}{}^{(t)}]]$ under *pk*. The transmitted message $M_i^{(t)} = (TagID_i, \{[[\hat{g}_{i,k}{}^{(t)}]]\}_{k=1}{}^{d})$ contains only the anonymous tag and encrypted encoded coordinates. The inset summarizes the local descent direction and the flatten-encode-encrypt pipeline.

Algorithm 2 Client-Side Local Training and Encrypted Gradient Submission

Require: Client C_i with data $\mathcal{D}_i$, round model $\boldsymbol{\theta}^{(t)}$, public key pk, scale s, epochs E, batch size B, learning rate η

Ensure: Encrypted submission $\mathcal{M}_i^{(t)}$

$\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}^{(t)}$
$\mathbf{g}_i^{(t)} \leftarrow \mathbf{0}$, $m_i \leftarrow 0$
for $e \leftarrow 1$ **to** E **do**
  for all mini-batches $\mathcal{B} \subset \mathcal{D}_i$ of size B **do**
    $\mathbf{h} \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}_i}; \mathcal{B})$
    $\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{h}$, $m_i \leftarrow m_i + 1$
    $\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}_i - \eta \mathbf{h}$
  end for
end for
$\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} / \max(m_i, 1)$
Flatten $\mathbf{g}_i^{(t)}$ into coordinates $\left(g_{i,1}^{(t)}, \ldots, g_{i,d}^{(t)}\right)$
for $k \leftarrow 1$ **to** d **do**
  $\widehat{g}_{i,k}^{(t)} \leftarrow \textsc{Encode}_s\left(g_{i,k}^{(t)}\right)$
  $[\![g_{i,k}^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}\left(\widehat{g}_{i,k}^{(t)}\right)$
end for
$\mathcal{M}_i^{(t)} \leftarrow \left(\text{TagID}_i, \{[\![g_{i,k}^{(t)}]\!]\}_{k=1}^{d}\right)$
Send $\mathcal{M}_i^{(t)}$ to the FIP Verifier

5.3.2 Phase II: Homomorphic FIP Score Computation

The Verifier computes the homomorphic inner product of each encrypted update with plaintext reference $\mathbf{R}^{(t)}$ and asks the TKA to decrypt only the scalar FIP score $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$. These scores drive the Phase III audit. Algorithm 3 gives the computation.

FIGURE · ALGORITHM 3

Homomorphic Functional Inner Product Score Computation

A geometric view of server-side encrypted scoring — the FIP Verifier combines encrypted encoded client coordinates with the encoded trusted reference, decrypts only the scalar accumulator through the TKA, decodes the signed fixed-point scalar score, and returns the round score map $\mathcal{S}^{(t)}$.

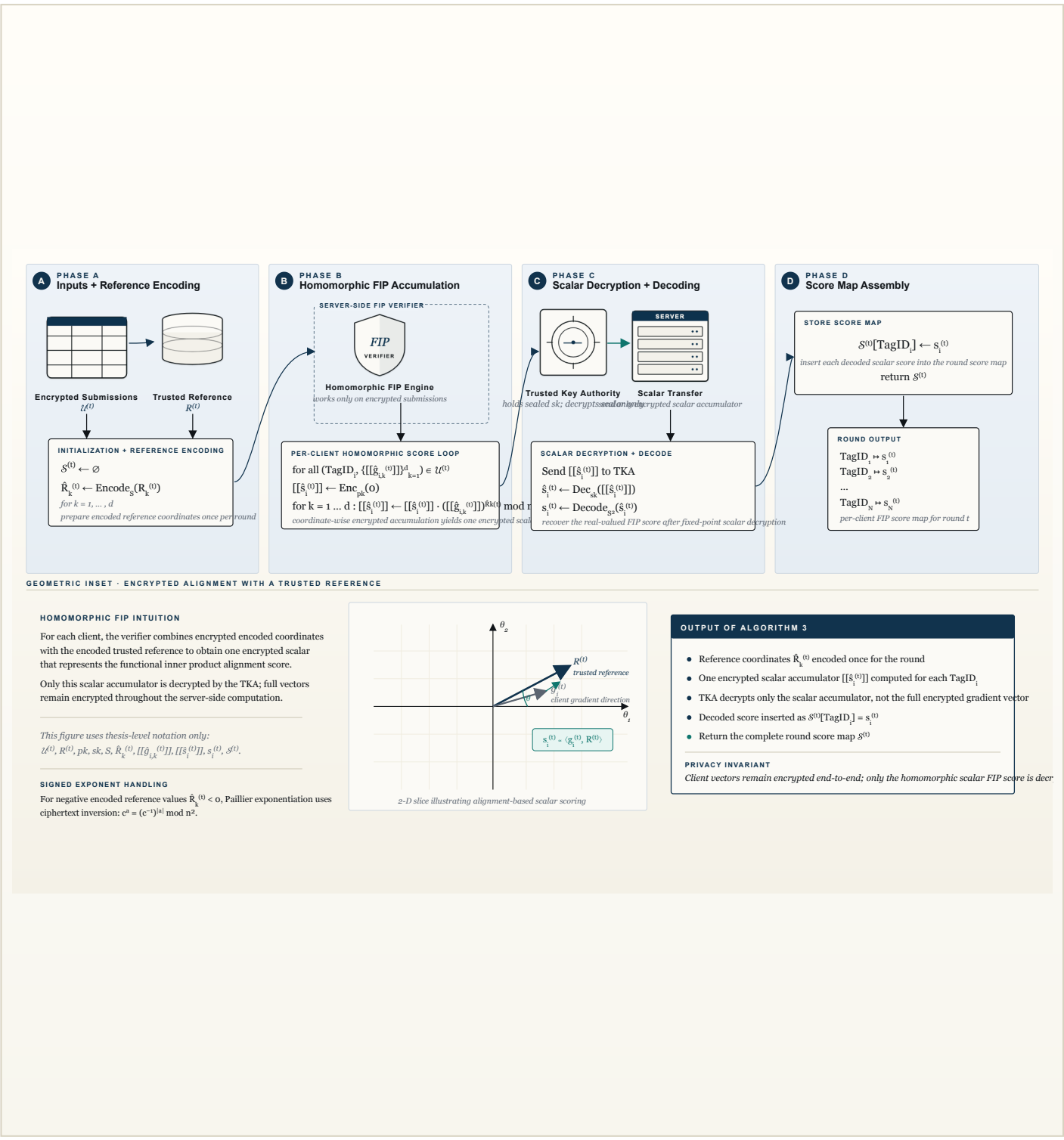


Figure 3. Algorithm 3. The FIP Verifier receives the encrypted submission set $\mathcal{U}^{(t)}$ and trusted reference $R^{(t)}$. It initializes the score map $\mathcal{S}^{(t)} = \varnothing$, encodes the reference coordinates as $\hat{R}_k^{(t)}$, and for each client accumulates an encrypted scalar inner-product score $[[\hat{s}_i^{(t)}]]$ using Paillier homomorphism. Only this scalar accumulator is sent to the TKA for decryption; the TKA returns $\hat{s}_i^{(t)}$, which is decoded as $s_i^{(t)}$ using $\text{Decode}_{S^2}(\cdot)$. The verifier stores the result as $\mathcal{S}^{(t)}[TagID_i] \leftarrow s_i^{(t)}$. The inset illustrates alignment-based scalar scoring with respect to the trusted reference direction.

Algorithm 3 Homomorphic Functional Inner Product Score Computation

Require: Encrypted submissions $\mathcal{U}^{(t)}$, reference $\mathbf{R}^{(t)}$, public key pk, secret key sk held by TKA, scale s

Ensure: FIP score map $\mathcal{S}^{(t)}$

$\mathcal{S}^{(t)} \leftarrow \emptyset$

Encode reference coordinates $\widehat{R}_k^{(t)} \leftarrow \textsc{Encode}_s\left(R_k^{(t)}\right)$ for $k = 1, \ldots, d$

for all $\left(\mathrm{TagID}_i, \{[\![g_{i,k}^{(t)}]\!]\}_{k=1}^{d}\right) \in \mathcal{U}^{(t)}$ **do**

  $[\![\widehat{s}_i^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}(0)$

  for $k \leftarrow 1$ **to** d **do**

    $[\![\widehat{s}_i^{(t)}]\!] \leftarrow [\![\widehat{s}_i^{(t)}]\!] \cdot \left([\![g_{i,k}^{(t)}]\!]\right)^{\widehat{R}_k^{(t)}} \bmod n^2$

  end for

  Send $[\![\widehat{s}_i^{(t)}]\!]$ to TKA for scalar decryption

  $\widehat{s}_i^{(t)} \leftarrow \textsc{Dec}_{sk}\left([\![\widehat{s}_i^{(t)}]\!]\right)$

  $s_i^{(t)} \leftarrow \textsc{Decode}_{s^2}\left(\widehat{s}_i^{(t)}\right)$

  $\mathcal{S}^{(t)}[\mathrm{TagID}_i] \leftarrow s_i^{(t)}$

end for

return $\mathcal{S}^{(t)}$

5.3.3 Phase III: Two-Stage Geometric Audit

The audit uses exactly two checks. The *direction filter* rejects non-positive FIP scores, and the *magnitude window* accepts only surviving scores within $\tau_{\min}^{(t)} \leq s_i^{(t)} \leq \tau_{\max}^{(t)}$. The lower bound catches weak positive updates; the upper bound catches overly large aligned updates. Algorithm 4 gives the complete procedure.

FIGURE · ALGORITHM 4

Two-Stage Geometric Audit for Malicious Client Detection

A geometric view of the audit logic — Stage 1 rejects directionally weak or negatively aligned clients using εdir, while Stage 2 applies robust median/MAD magnitude screening to the provisional pass set and returns accepted encrypted submissions $\mathcal{A}^{(t)}$ and rejected tags $\mathcal{D}^{(t)}$.

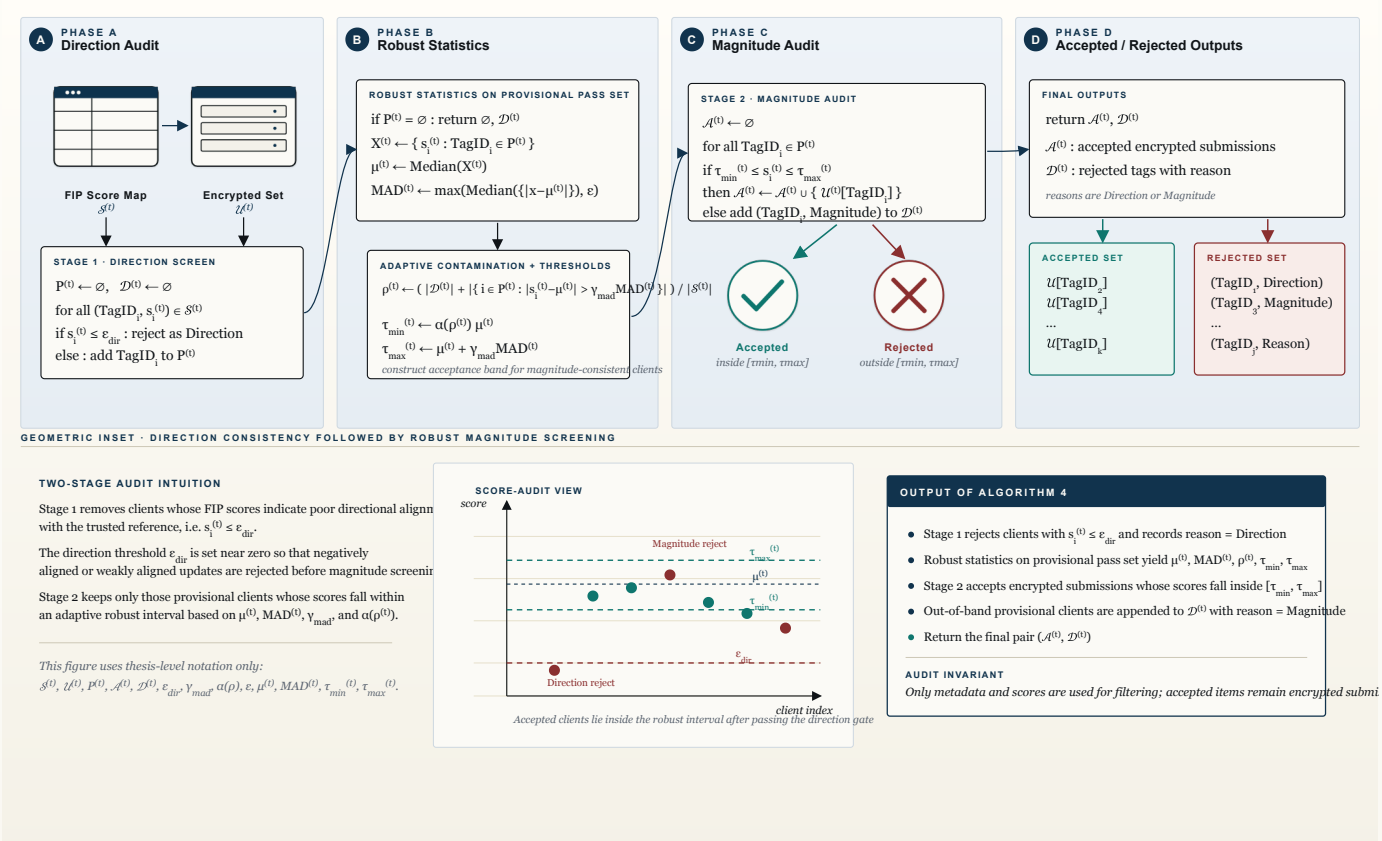


Figure 4. Algorithm 4. The verifier takes the FIP score map $\mathcal{S}^{(t)}$ and encrypted submission set $\mathcal{U}^{(t)}$. In Stage 1, it rejects clients with $s_i^{(t)} \le \varepsilon_{dir}$ as directionally suspicious and forms the provisional pass set $P^{(t)}$. In Stage 2, it computes the robust statistics $\mu^{(t)}$ and $MAD^{(t)}$, estimates the contamination rate $\rho^{(t)}$, and constructs the adaptive interval $[\tau_{min}^{(t)}, \tau_{max}^{(t)}]$. Encrypted submissions whose scores fall inside this band are retained in the accepted set $\mathcal{A}^{(t)}$, while the others are added to the rejected tag set $\mathcal{D}^{(t)}$ with reason Direction or Magnitude. The inset visualizes this two-stage screening in score space.

Algorithm 4 Two-Stage Geometric Audit for Malicious Client Detection

Require: FIP scores $\mathcal{S}^{(t)}$, encrypted submissions $\mathcal{U}^{(t)}$, direction threshold ϵ_{dir}, MAD multiplier γ_{mad}, floor map $\alpha(\rho)$, stability floor ε

Ensure: Accepted encrypted set $\mathcal{A}^{(t)}$, rejected tag set $\mathcal{D}^{(t)}$

$\mathcal{P}^{(t)} \leftarrow \emptyset$, $\mathcal{D}^{(t)} \leftarrow \emptyset$
for all $\left(\text{TagID}_i, s_i^{(t)}\right) \in \mathcal{S}^{(t)}$ **do**
  if $s_i^{(t)} \leq \epsilon_{\text{dir}}$ **then**
    $\mathcal{D}^{(t)} \leftarrow \mathcal{D}^{(t)} \cup \{(\text{TagID}_i, \textsc{Direction})\}$
  else
    $\mathcal{P}^{(t)} \leftarrow \mathcal{P}^{(t)} \cup \{\text{TagID}_i\}$
  end if
end for
if $\mathcal{P}^{(t)} = \emptyset$ **then**
  return $\emptyset, \mathcal{D}^{(t)}$
end if
$\mathcal{X}^{(t)} \leftarrow \{s_i^{(t)} : \text{TagID}_i \in \mathcal{P}^{(t)}\}$
$\mu^{(t)} \leftarrow \textsc{Median}\left(\mathcal{X}^{(t)}\right)$
$\text{MAD}^{(t)} \leftarrow \max\left(\textsc{Median}\left(\{|x - \mu^{(t)}| : x \in \mathcal{X}^{(t)}\}\right), \varepsilon\right)$
$\rho^{(t)} \leftarrow \frac{|\mathcal{D}^{(t)}| + \left|\{i \in \mathcal{P}^{(t)} : |s_i^{(t)} - \mu^{(t)}| > \gamma_{\text{mad}} \text{MAD}^{(t)}\}\right|}{|\mathcal{S}^{(t)}|}$
$\tau_{\min}^{(t)} \leftarrow \alpha\left(\rho^{(t)}\right) \mu^{(t)}$
$\tau_{\max}^{(t)} \leftarrow \mu^{(t)} + \gamma_{\text{mad}} \text{MAD}^{(t)}$
$\mathcal{A}^{(t)} \leftarrow \emptyset$
for all $\text{TagID}_i \in \mathcal{P}^{(t)}$ **do**
  if $\tau_{\min}^{(t)} \leq s_i^{(t)} \leq \tau_{\max}^{(t)}$ **then**
    $\mathcal{A}^{(t)} \leftarrow \mathcal{A}^{(t)} \cup \{\mathcal{U}^{(t)}[\text{TagID}_i]\}$
  else
    $\mathcal{D}^{(t)} \leftarrow \mathcal{D}^{(t)} \cup \{(\text{TagID}_i, \textsc{Magnitude})\}$
  end if
end for
return $\mathcal{A}^{(t)}, \mathcal{D}^{(t)}$

5.3.4 Phase IV: Secure Aggregation

The Aggregation Server homomorphically adds only accepted ciphertexts and asks the TKA to decrypt the aggregate vector, never an individual update.

Rejected updates stay encrypted and discarded; accepted updates appear only through their pooled sum.

5.3.5 Phase V: Reference Update and Model Distribution

The final phase updates the model from the averaged accepted gradient and refreshes the reference by an exponential-moving average with momentum β. Normalisation keeps the next round's FIP scores directional; if no client is accepted, the model and reference are preserved. Algorithm 5 combines secure aggregation, model update, and reference update.

FIGURE · ALGORITHM 5

Secure Aggregation and Momentum-Based Reference Update

A geometric view of the final round update — accepted encrypted coordinates are homomorphically aggregated, only aggregate coordinates are decrypted by the TKA, the averaged global gradient updates the model, and a momentum-normalized trusted reference $R^{(t+1)}$ is stored for the next round.

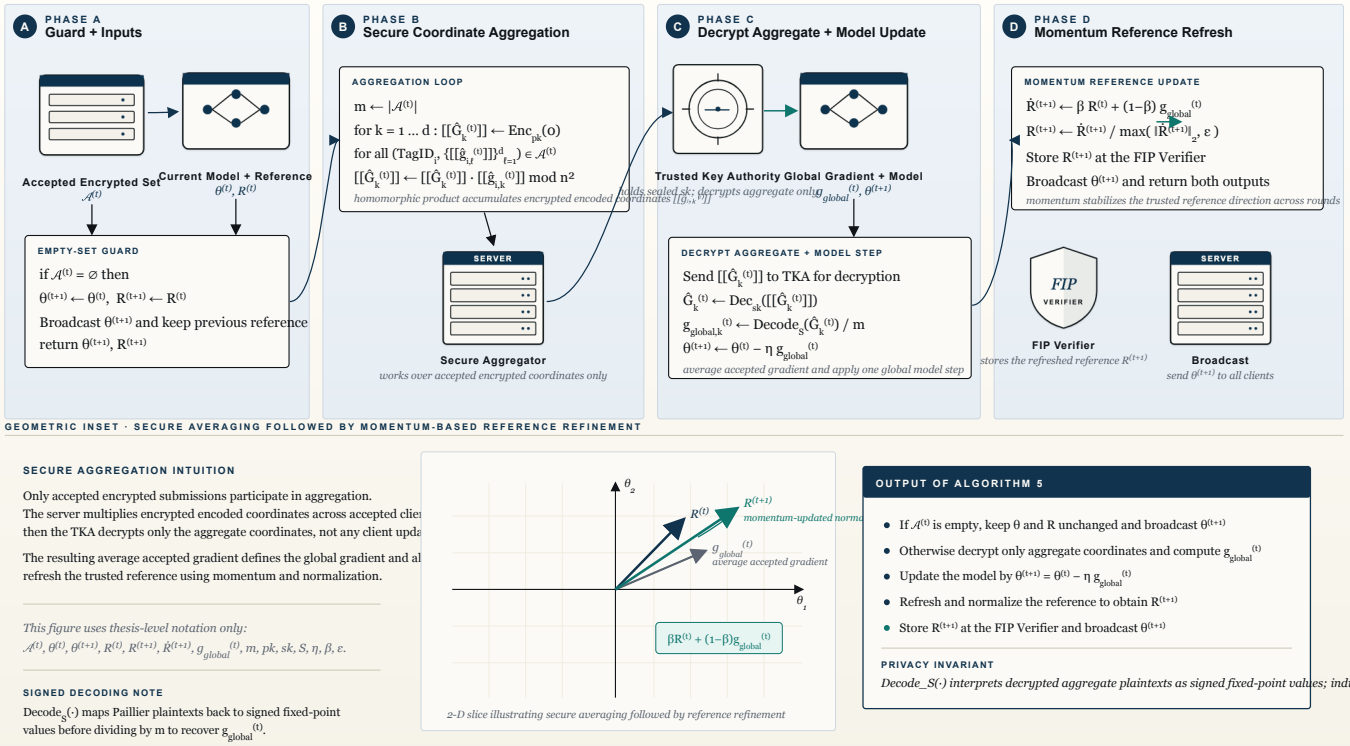


Figure 5. Algorithm 5. If the accepted encrypted set $\mathcal{A}^{(t)}$ is empty, the server keeps $\theta^{(t+1)} \leftarrow \theta^{(t)}$ and $R^{(t+1)} \leftarrow R^{(t)}$, broadcasts the unchanged model, and returns. Otherwise, for each coordinate k, the server homomorphically aggregates the accepted encrypted encoded values into $[[\hat{G}_k^{(t)}]]$ and sends only these aggregate ciphertexts to the TKA. After decryption and signed fixed-point decoding, the average accepted gradient $g_{global}^{(t)}$ is recovered and used to update the model by $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta g_{global}^{(t)}$. The trusted reference is then refreshed as $\dot{R}^{(t+1)} \leftarrow \beta R^{(t)} + (1-\beta) g_{global}^{(t)}$ and normalized to $R^{(t+1)}$ using $\max(\|\dot{R}^{(t+1)}\|_2, \varepsilon)$. The inset illustrates secure averaging followed by momentum-based reference refinement.

Algorithm 5 Secure Aggregation and Momentum-Based Reference Update

Require: Accepted encrypted set $\mathcal{A}^{(t)}$, model $\boldsymbol{\theta}^{(t)}$, reference $\mathbf{R}^{(t)}$, public key pk, secret key sk held by TKA, scale s, learning rate η, momentum β, stability floor ε

Ensure: Updated model $\boldsymbol{\theta}^{(t+1)}$, updated reference $\mathbf{R}^{(t+1)}$

if $\mathcal{A}^{(t)} = \emptyset$ **then**
  $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)}$, $\mathbf{R}^{(t+1)} \leftarrow \mathbf{R}^{(t)}$
  Broadcast $\boldsymbol{\theta}^{(t+1)}$ and keep the previous reference
  return $\boldsymbol{\theta}^{(t+1)}, \mathbf{R}^{(t+1)}$
end if
$m \leftarrow |\mathcal{A}^{(t)}|$
for $k \leftarrow 1$ **to** d **do**
  $[\![\widehat{G}_k^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}(0)$
  for all $\left(\text{TagID}_i, \{[\![g_{i,\ell}^{(t)}]\!]\}_{\ell=1}^{d}\right) \in \mathcal{A}^{(t)}$ **do**
    $[\![\widehat{G}_k^{(t)}]\!] \leftarrow [\![\widehat{G}_k^{(t)}]\!] \cdot [\![g_{i,k}^{(t)}]\!] \bmod n^2$
  end for
  Send $[\![\widehat{G}_k^{(t)}]\!]$ to TKA for aggregate decryption
  $\widehat{G}_k^{(t)} \leftarrow \textsc{Dec}_{sk}\left([\![\widehat{G}_k^{(t)}]\!]\right)$
  $g_{\text{global},k}^{(t)} \leftarrow \textsc{Decode}_s\left(\widehat{G}_k^{(t)}\right)/m$
end for
$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \eta \mathbf{g}_{\text{global}}^{(t)}$
$\widetilde{\mathbf{R}}^{(t+1)} \leftarrow \beta \mathbf{R}^{(t)} + (1-\beta)\mathbf{g}_{\text{global}}^{(t)}$
$\mathbf{R}^{(t+1)} \leftarrow \widetilde{\mathbf{R}}^{(t+1)} / \max\left(\left\|\widetilde{\mathbf{R}}^{(t+1)}\right\|_2, \varepsilon\right)$
Store $\mathbf{R}^{(t+1)}$ at the FIP Verifier and broadcast $\boldsymbol{\theta}^{(t+1)}$
return $\boldsymbol{\theta}^{(t+1)}, \mathbf{R}^{(t+1)}$

5.4 Theoretical Analysis

The three guarantees below correspond to the goals of Section 4.6.

5.4.1 Gradient Privacy

Theorem 5.1 (Gradient Privacy)**.** *In every round t, the FIP Verifier reveals exactly one scalar per client, namely the projection $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle \in \mathbb{R}$. For a*

gradient of dimension $d \gg 1$, recovering $\mathbf{g}_i^{(t)}$ from $s_i^{(t)}$ is information-theoretically impossible: the linear system $\mathbf{R}^{(t)\top}\mathbf{g}_i^{(t)} = s_i^{(t)}$ is under-determined and admits a $(d-1)$-dimensional affine solution space. Equivalently, the mutual information satisfies $I(\mathbf{g}_i^{(t)}; s_i^{(t)}) \le H(s_i^{(t)}) \ll H(\mathbf{g}_i^{(t)})$.

Proof. The FIP Verifier's only operation on ciphertext $[\![\mathbf{g}_i^{(t)}]\!]$ is the homomorphic inner product of (4.8), which yields $[\![s_i^{(t)}]\!]$. Given $s_i^{(t)}$ and $\mathbf{R}^{(t)}$, the preimage of $\mathbf{g} \mapsto \langle \mathbf{g}, \mathbf{R}^{(t)} \rangle$ is a $(d-1)$-dimensional affine hyperplane. Gradients differing by a vector orthogonal to $\mathbf{R}^{(t)}$ are indistinguishable from the scalar alone, and the data-processing inequality gives $I(\mathbf{g}_i^{(t)}; s_i^{(t)}) \le H(s_i^{(t)}) \ll H(\mathbf{g}_i^{(t)})$. □

5.4.2 Attack Resilience

Theorem 5.2 (Attack Resilience)**.** *Any malicious update $\mathbf{g}_{adv}$ accepted by the FIP-FL filter in round t must satisfy simultaneously*

$$\langle \mathbf{g}_{adv}, \mathbf{R}^{(t)} \rangle > 0 \qquad \textit{(direction constraint)}, \tag{5.1}$$

$$\tau_{\min}^{(t)} \le \langle \mathbf{g}_{adv}, \mathbf{R}^{(t)} \rangle \le \tau_{\max}^{(t)} \qquad \textit{(magnitude constraint)}, \tag{5.2}$$

where $\tau_{\min}^{(t)}$ and $\tau_{\max}^{(t)}$ are computed from the round-wise positive-score distribution. The two constraints jointly rule out direction-reversal, scaling-to-zero, and destabilising large-norm attacks while keeping the audit independent of any external auditor.

Proof. The geometric audit first applies the non-positive direction check and then the $\tau_{\min}^{(t)}/\tau_{\max}^{(t)}$ magnitude window. An adversary failing either clause is rejected. A direct poisoning gradient $\mathbf{g}_{\text{adv}} \approx -\mathbf{g}_{\text{honest}}$ gives a non-positive score, a near-zero scaling attack falls below $\tau_{\min}^{(t)}$, and a destabilising large-norm attack exceeds $\tau_{\max}^{(t)} = \mu^{(t)} + 5 \cdot \text{MAD}^{(t)}$. □

5.4.3 Convergence Guarantee

Theorem 5.3 (Descent under L-Smoothness)**.** *Let $F(\mathbf{w})$ be L-smooth and let $\Delta_{global}^{(t)}$ be the mean of the accepted updates in round t. If the accepted aggregate*

is descent-aligned with the true gradient,

$$\langle \nabla F(\mathbf{w}^{(t)}),\, \Delta_{global}^{(t)} \rangle > 0, \tag{5.3}$$

then any learning rate

$$0 < \eta < \frac{2\langle \nabla F(\mathbf{w}^{(t)}),\, \Delta_{global}^{(t)} \rangle}{L \|\Delta_{global}^{(t)}\|_2^2} \tag{5.4}$$

gives a strict one-round loss decrease:

$$F(\mathbf{w}^{(t+1)}) < F(\mathbf{w}^{(t)}). \tag{5.5}$$

Consequently, if this alignment holds in expectation over accepted rounds, the expected global loss decreases monotonically.

Proof. The L-smoothness descent lemma gives

$$F(\mathbf{w}^{(t)} - \eta \Delta_{\text{global}}^{(t)}) \leq F(\mathbf{w}^{(t)}) - \eta \langle \nabla F(\mathbf{w}^{(t)}), \Delta_{\text{global}}^{(t)} \rangle + \frac{L\eta^2}{2} \|\Delta_{\text{global}}^{(t)}\|_2^2. \tag{5.6}$$

Under the stated alignment condition, the right-hand side is strictly below $F(\mathbf{w}^{(t)})$ whenever η satisfies the stated bound. The FIP-FL filter supports this condition by admitting only updates with positive reference projection and bounded scalar magnitude; the theorem states the resulting descent requirement explicitly rather than assuming the reference is perfectly equal to the true gradient. □

Together, the guarantees define the design checklist for the experiments: the scalar FIP score must hide the full gradient, reject harmful or over-scaled directions, and leave the decrypted accepted aggregate descent-oriented under the stated smoothness assumption.

5.5 Hyperparameters

Table 5.1 lists the hyperparameters used in Chapter 7. Defaults follow the SenSys 2026 poster and stay fixed unless explicitly noted.

Table 5.1: FIP-FL hyperparameters.

Parameter	**Value**	**Description**
N	10	Number of participating clients
R	300	Total communication rounds
B	50	Local batch size
η	0.01	Learning rate
E	1	Local epochs per round
λ	1024	Paillier key size (bits)
s	10,000	Fixed-point scale factor
ϵ_{dir}	0.0	Direction-filter threshold for positive FIP alignment
γ_{mad}	5.0	MAD multiplier for the magnitude upper cap
ε	10^{-6}	Numerical stability floor
Adaptive magnitude-window ratios keyed by estimated attack rate ρ		
$\rho < 0.15$	0.55	Floor ratio at low attack rates
$0.15 \leq \rho < 0.25$	0.50	Floor ratio at medium attack rates
$0.25 \leq \rho < 0.35$	0.40	Floor ratio at high attack rates
$\rho \geq 0.35$	0.30	Floor ratio at extreme attack rates
Reference momentum		
β	0.80	Momentum weight for the previous reference direction

6 Experimental Setup

This chapter defines the empirical protocol for FIP-FL: datasets, partitions, attacks, metrics, implementation stack, and hardware.

6.1 Datasets

FIP-FL is evaluated on one vision benchmark and one intrusion-detection benchmark, summarized in Table 6.1.

Table 6.1: Datasets used in the evaluation of FIP-FL.

Dataset	Modality	Classes	Features	Samples
MNIST	Image	10	784	60,000/10,000
KDDCup99	Tabular	5	41	$\approx$ 494K (10%)

MNIST. MNIST contains 60,000 training and 10,000 test grayscale images of size 28×28, labelled over digits $\{0, 1, \ldots, 9\}$. Each image is flattened to a 784-dimensional vector and normalized to the unit interval. MNIST is also used by Yazdinejad et al. [1], so it provides the direct comparison point with the auditor-based baseline.

KDDCup99. KDDCup99 is a network-intrusion detection dataset. The full corpus has about 4.9 million labelled connection records; this thesis uses the standard 10% subset, containing approximately 494,000 records. Each record has 41 traffic and protocol features, and labels are collapsed into five classes: Normal, DoS, Probe, R2L, and U2R. Categorical features are one-hot

encoded and all features are standardized. Its strong class imbalance makes it a useful stress test for Byzantine filtering under uneven class priors.

Model architecture. For both datasets I use a softmax logistic-regression classifier, matching the SenSys 2026 poster and the auditor-based baseline. Its parameter dimension is $d = (\text{input\_dim}) \cdot C + C$, which keeps Paillier encryption tractable at $\lambda = 1024$ bits, and its convex L-smooth loss matches the assumptions used in Theorem 5.3.

6.2 Data Partitioning

Each dataset is split into $N = 10$ disjoint client shards under two regimes. In the IID regime, the training set is shuffled and dealt into equal shards, so each client receives approximately $|\mathcal{D}_{\text{train}}|/N$ samples and the expected label distribution matches the global distribution.

In the non-IID regime, I use label skew. For a C-class task, client i receives a primary label set $\mathcal{L}_i \subset \{0, 1, \ldots, C-1\}$ of size $\lceil C/2 \rceil$, with overlap $\lfloor C/4 \rfloor$ between consecutive clients. Samples are assigned so that 80% of a client's data comes from $\mathcal{L}_i$ and the remaining 20% is drawn from the full training set, preserving a small presence of every class. This matches the SenSys 2026 evaluation protocol and closely follows the non-IID setup of Yazdinejad et al. [1]. The setting is intentionally difficult: honest clients can point toward different but still useful descent directions, so the filter must tolerate heterogeneity rather than simply reward collinearity.

6.3 Attack Models

The evaluation uses two pre-training label-flipping attacks; neither requires cryptographic knowledge of FIP-FL.

Untargeted label flipping. A malicious client relabels each local sample by a fixed cyclic offset:

$$y_k^{(i,\,\text{mal})} \;=\; (y_k^{(i)} + \kappa) \bmod C. \tag{6.1}$$

For MNIST I use $\kappa = 5$, giving a no-fixed-point digit permutation; for KDDCup99

I use $\kappa = 2$ because the task has five classes. The objective is to reduce overall accuracy by pulling local gradients toward an intentionally wrong classification objective.

Targeted label flipping. A malicious client changes only a source class c_s to a target class c_t:

$$y_k^{(i,\text{mal})} = \begin{cases} c_t & \text{if } y_k^{(i)} = c_s, \\ y_k^{(i)} & \text{otherwise.} \end{cases} \tag{6.2}$$

For MNIST I use $(c_s, c_t) = (0, 7)$; for KDDCup99 I use the analogous source-target pair from the dominant classes in the skew regime. This attack aims to damage one semantic class while leaving overall accuracy comparatively high, so it is measured primarily through target-class accuracy.

Malicious-client rates. For every dataset, partition, and attack type, the malicious-client fraction is swept over $f/N \in \{10\%, 20\%, 30\%, 50\%\}$. The 50% case is the headline stress test near the limit of majority-based aggregation.

6.4 Evaluation Metrics

Five metrics are reported throughout: three for held-out utility and two for verifier-side detection quality.

Target accuracy (T_{acc}). Accuracy restricted to test samples from the targeted source class c_s:

$$T_{\text{acc}} = \frac{|\{(x,y) \in \mathcal{D}_{\text{test}} : y = c_s,\ \hat{f}(x) = c_s\}|}{|\{(x,y) \in \mathcal{D}_{\text{test}} : y = c_s\}|}. \tag{6.3}$$

A successful targeted attack drives this value down; a successful defence keeps it high. For untargeted attacks, class 0 is used as the fixed diagnostic reference class.

Other-label accuracy (O_{acc}). Accuracy on all test samples outside the targeted source class:

$$O_{\text{acc}} = \frac{|\{(x,y) \in \mathcal{D}_{\text{test}} : y \neq c_s,\ \hat{f}(x) = y\}|}{|\{(x,y) \in \mathcal{D}_{\text{test}} : y \neq c_s\}|}. \tag{6.4}$$

This separates targeted damage from collateral loss on the rest of the task.

Overall accuracy (ACC). Standard test accuracy over all classes:

$$\text{ACC} = \frac{|\{(x, y) \in \mathcal{D}_{\text{test}} : \hat{f}(x) = y\}|}{|\mathcal{D}_{\text{test}}|}. \tag{6.5}$$

This is the primary utility metric for untargeted attacks.

True positive rate (TPR). Fraction of malicious clients rejected by the FIP-FL audit:

$$\text{TPR} = \frac{|\{i : \text{client } i \text{ is malicious and is rejected}\}|}{|\{i : \text{client } i \text{ is malicious}\}|}. \tag{6.6}$$

TPR measures detection sensitivity; TPR = 1.00 means no malicious client is accepted.

False positive rate (FPR). Fraction of honest clients rejected by the FIP-FL audit:

$$\text{FPR} = \frac{|\{i : \text{client } i \text{ is honest and is rejected}\}|}{|\{i : \text{client } i \text{ is honest}\}|}. \tag{6.7}$$

FPR measures the cost of filtering; all metrics are averaged over three independent seeds and reported with a standard deviation when it exceeds 0.5 percentage points.

6.5 Implementation and Hardware

Experiments are implemented in Python 3 using `NumPy` for linear algebra, `scikit-learn` for dataset loading and feature engineering, and `phe` for Paillier encryption with a 1024-bit modulus. The FIP Verifier and Aggregation Server run as separate Python processes connected through in-memory queues; the N clients are simulated as worker processes. Homomorphic encryption and homomorphic inner products are parallelized with `multiprocessing`.

Wall-clock measurements in Chapter 8 use a workstation with a 25-core CPU, 128 GB RAM, and a Linux-based operating system. The 13× speed-up cited in the abstract is measured on the Phase II homomorphic inner-product step against the naive single-core implementation.

7 Results and Analysis

This chapter reports the completed FIP-FL evaluation from the grid defined in Chapter 6. Tables give the relevant round-300 utility and detection metrics: overall accuracy (ACC), target-class accuracy (T_{acc}), other-label accuracy (O_{acc}), true positive rate (TPR), and false positive rate (FPR). Accuracies are percentages; TPR and FPR are proportions.

7.1 Non-IID Partition, Untargeted Attacks

This is the hardest untargeted setting: label skew already separates honest-client gradients, and malicious clients add label-flipped gradients that point away from the descent direction. Overall accuracy is the main utility metric, while TPR and FPR show the cost of filtering.

7.1.1 MNIST

Table 7.1: MNIST, non-IID partition, untargeted label-flipping attack.

AR	**ACC (%)**	$T_{\textbf{acc}}$ **(%)**	$O_{\textbf{acc}}$ **(%)**	**TPR**	**FPR**
10%	91.22	97.60	90.18	1.00	0.00
20%	91.02	97.87	89.94	1.00	0.00
30%	90.78	97.33	89.31	1.00	0.00
50%	91.10	97.51	89.91	1.00	0.20

7.1.2 KDDCup99

Table 7.2: KDDCup99, non-IID partition (Dirichlet $\alpha = 0.5$), untargeted label-flipping attack.

AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR	FPR
10%	99.71	99.50	84.62	1.00	0.00
20%	99.72	99.55	83.65	1.00	0.00
30%	99.69	99.45	85.58	1.00	0.14
50%	99.69	99.70	77.88	1.00	0.40

Observations. Across MNIST and KDDCup99, FIP-FL rejects every malicious update and preserves utility under non-IID untargeted poisoning. MNIST ACC stays within 90.78–91.22% with false rejection only at 50% attack rate, while KDDCup99 ACC remains above 99.6% even when FPR rises under the heaviest non-IID load.

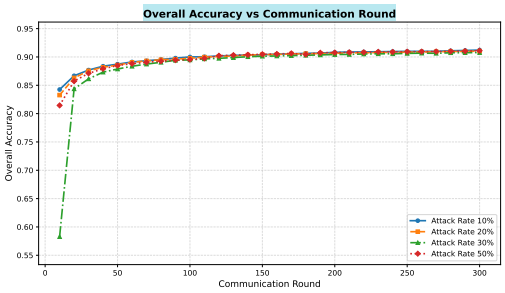
7.2 Non-IID Partition, Targeted Attacks

Targeted label flipping is subtler than untargeted poisoning because it tries to damage one source class while leaving global accuracy plausible. The primary diagnostic is therefore T_{acc}.

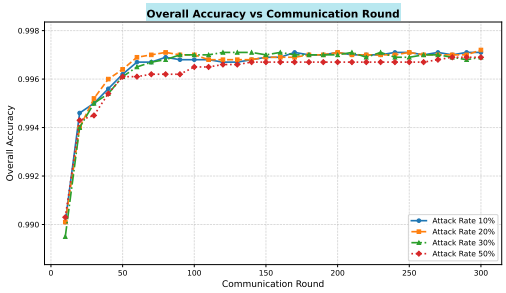
7.2.1 MNIST

Table 7.3: MNIST, non-IID, targeted label-flipping attack ($c_s = 0, c_t = 7$); SenSys 2026 headline targeted-accuracy cell.

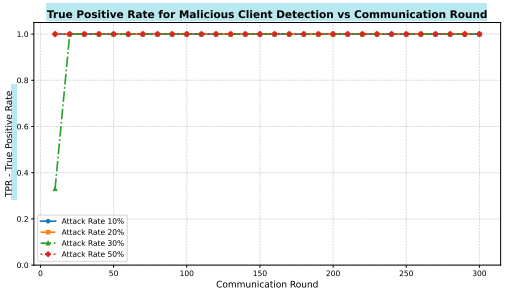
AR	Undefended (%)	Auditor-based (%)	FIP-FL (%)
50%	≈ 7.00	96.50	97.69



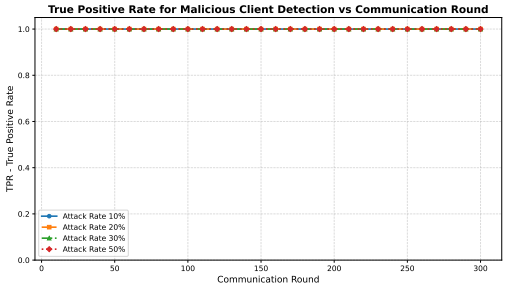
(a) MNIST ACC trajectory



(b) KDDCup99 ACC trajectory



(c) MNIST malicious-client TPR



(d) KDDCup99 malicious-client TPR

Figure 7.1: Non-IID untargeted attack dynamics over 300 communication rounds. Accuracy converges stably for both datasets, while the direction/magnitude audit reaches perfect malicious-client detection by the final round.

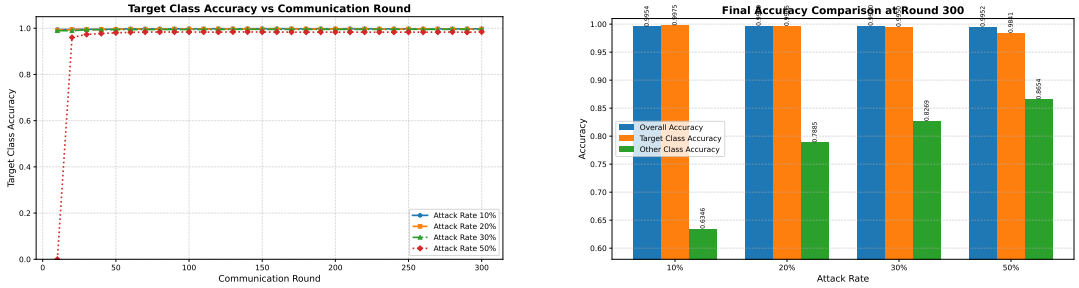
7.2.2 KDDCup99

Table 7.4: KDDCup99, non-IID partition (2 shards/client), targeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	99.54	99.75	63.46	1.00	0.00
20%	99.69	99.65	78.85	1.00	0.00
30%	99.70	99.50	82.69	1.00	0.00
50%	99.52	98.41	86.54	1.00	0.00

Observations. The MNIST cell is the accepted-poster headline result: at 50% malicious clients, FIP-FL recovers T_{acc} from roughly 7% undefended to 97.69%, above the 96.5% auditor-based result of Yazdinejad et al. [1]. KDDCup99 shows the same detection pattern across the full sweep, with no false positives and

$T_{\text{acc}} = 98.41\%$ even at 50% malicious clients.



(a) Target-class accuracy trajectory (b) Round-300 accuracy comparison

Figure 7.2: KDDCup99 non-IID targeted attack results. The target-class trajectory remains high through training, and the round-300 comparison shows that overall and target-class accuracy stay stable even as the malicious-client fraction increases.

7.3 IID Partition, Untargeted Attacks

IID partitions are included as a sanity check because honest clients are statistically similar and the FIP score distribution is correspondingly tight.

7.3.1 MNIST

Table 7.5: MNIST, IID partition, untargeted label-flipping attack.

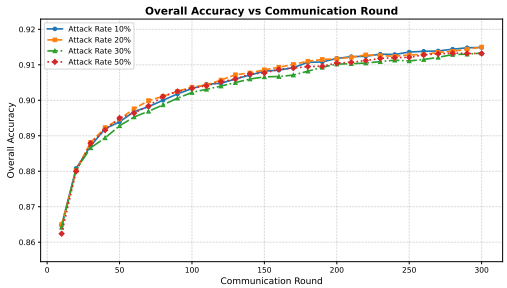
AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	91.49	97.51	90.40	1.00	0.00
20%	91.50	97.51	90.39	1.00	0.00
30%	91.33	97.42	90.28	1.00	0.00
50%	91.32	97.42	90.22	1.00	0.00

7.3.2 KDDCup99

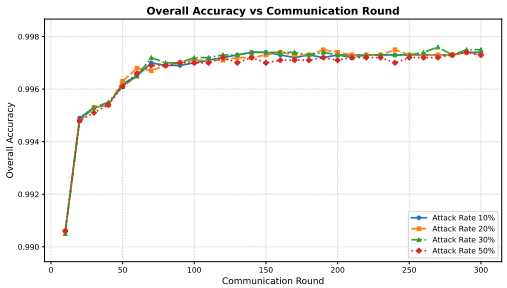
Table 7.6: KDDCup99, IID partition, untargeted label-flipping attack.

AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR	FPR
10%	99.74	99.60	85.58	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.75	99.60	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

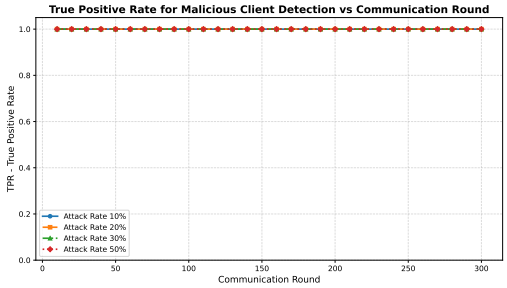
Observations. In the IID untargeted regime, detection is exact for both datasets: TPR is 1.00 and FPR is 0.00 throughout. MNIST ACC changes by less than 0.2 percentage points, and KDDCup99 ACC, T_{acc}, and O_{acc} vary by at most 0.05 percentage points.



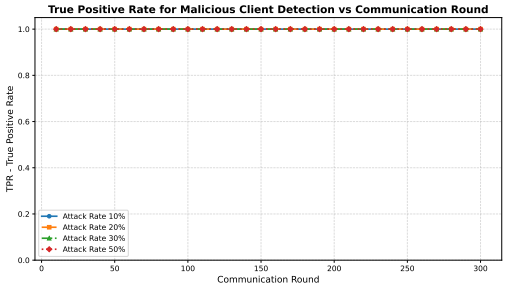
(a) MNIST ACC trajectory



(b) KDDCup99 ACC trajectory



(c) MNIST malicious-client TPR



(d) KDDCup99 malicious-client TPR

Figure 7.3: IID untargeted attack dynamics over 300 communication rounds. Both datasets show stable convergence, and the audit maintains perfect malicious-client detection without false positives at the reported final round.

7.4 IID Partition, Targeted Attacks

7.4.1 MNIST

Table 7.7: MNIST, IID partition, targeted label-flipping attack ($c_s = 0, c_t = 7$).

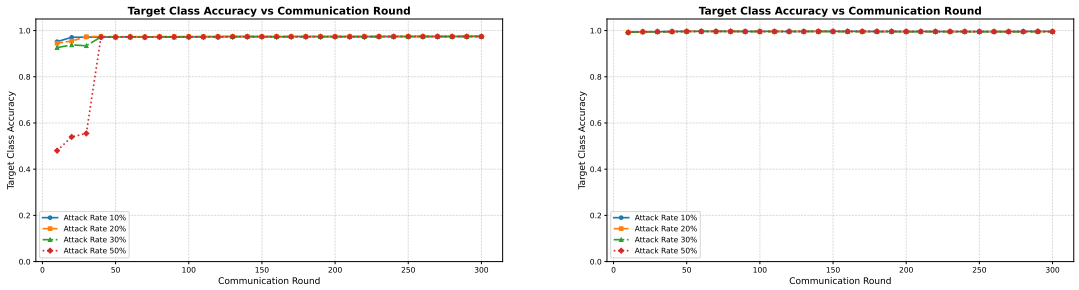
AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR	FPR
10%	91.62	97.51	90.57	1.00	0.00
20%	91.42	97.42	90.35	1.00	0.00
30%	91.32	97.42	90.22	1.00	0.00
50%	91.43	97.42	90.35	1.00	0.00

7.4.2 KDDCup99

Table 7.8: KDDCup99, IID partition, targeted label-flipping attack.

AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR	FPR
10%	99.73	99.60	84.62	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.72	99.50	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

Observations. The IID targeted case matches the IID untargeted pattern: perfect detection, zero false rejection, and stable target-class accuracy. MNIST keeps $T_{\text{acc}} \geq 97.42\%$ across all attack rates, while KDDCup99 keeps ACC at 99.72–99.73% and $T_{\text{acc}} \geq 99.50\%$.



(a) MNIST target-class accuracy

(b) KDDCup99 target-class accuracy

Figure 7.4: IID targeted attack target-class accuracy over 300 communication rounds. In both datasets, the target class recovers to a stable high-accuracy regime by the final round.

7.5 Aggregate Findings Across the Grid

The completed grid supports three conclusions. First, every completed row has TPR = 1.00, including the 50% malicious-client cases. Second, false rejection appears only under heavy non-IID untargeted load, where honest gradients are already spread by label skew; even there, ACC remains stable. Third, targeted attacks are detected as reliably as untargeted attacks, showing that a one-dimensional FIP score against the trusted reference is sufficient for the evaluated poisoning settings. Chapter 8 compares these results with the auditor-based baseline.

8 Comparative Analysis

This chapter compares FIP-FL with the Byzantine-resilient FL defences surveyed in Chapter 3, especially the auditor-based baseline of Yazdinejad et al. [1]. The comparison focuses on privacy leakage, computation, communication, non-IID robustness, engineering cost, and the canonical MNIST non-IID targeted benchmark.

8.1 Privacy

The auditor-based scheme decrypts cluster-level statistics in each round: a projected Gaussian-mixture model and the distance statistics used for filtering. Even when individual gradients are not reconstructed, those cluster means and covariances reveal structural information about the client population. In non-IID deployments, such structure can correlate with demographics, hospital specialization, keyboard language, or other sensitive client attributes.

FIP-FL reveals only one scalar per client per round, the FIP score $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$. No cluster means, covariance matrices, projection coefficients, or pairwise distances are decrypted. Thus the privacy improvement is categorical: the verifier sees one alignment score, while the model update is still computed only from an aggregate.

8.2 Computational Complexity

Table 8.1: Per-round computational cost. N: clients; $|\mathcal{A}|$: accepted clients; d: model dimension; B: batch size; E: local epochs.

Entity	**Auditor-based [1]**	**FIP-FL**
Client	$\mathcal{O}(dBE)$	$\mathcal{O}(dBE)$
Verifier / Auditor	$\mathcal{O}(NKD + Nd\log d) + \mathcal{O}(d^3)$	$\mathcal{O}(Nd + N\log N)$
Aggregation Server	$\mathcal{O}(dN\log N)$	$\mathcal{O}(\lvert\mathcal{A}\rvert d)$

The client cost is the same because both schemes run local SGD. The difference is at verification and aggregation: the auditor pays for projection, mixture fitting, and covariance inversion, while FIP-FL uses N homomorphic inner products plus median/MAD statistics. The aggregation server then sums only accepted ciphertexts, giving $\mathcal{O}(|\mathcal{A}|d)$ work instead of sorting or preprocessing all N updates.

8.3 Communication Complexity

Table 8.2: Per-round communication cost by directed link.

Link	**Auditor-based [1]**	**FIP-FL**
Client $\rightarrow$ Verifier	$\mathcal{O}(Nd)$	$\mathcal{O}(Nd)$
Verifier $\rightarrow$ Server	$\mathcal{O}(L(Z+N))$	$\mathcal{O}(\lvert\mathcal{A}\rvert d)$
Server $\rightarrow$ Clients	$\mathcal{O}(d)$	$\mathcal{O}(d)$

The initial client-to-verifier payload is identical: encrypted updates must be submitted in both designs. FIP-FL saves communication after verification because rejected ciphertexts are never forwarded to the aggregation server, and it avoids the extra auditor communication step used by auditor-based designs.

8.4 Robustness and Engineering Trade-Offs

The auditor-based detector relies on the honest-client cluster being tighter and more populous than the malicious cluster, an assumption weakened by label skew. FIP-FL instead tests whether each update is directionally aligned with the trusted reference and lies inside the adaptive magnitude window, so the admissibility test remains one-dimensional even when honest clients are heterogeneous.

FIP-FL also limits recovery cascades after difficult rounds. Only updates passing both checks enter the aggregate, and the next reference direction is updated by exponential moving average rather than full replacement. The Phase II FIP computations are independent across clients, giving a 13× wall-clock speed-up on the 25-core workstation described in Section 6.5; the single-reference design also avoids the extra homomorphic inner products required by a multi-reference cone.

8.5 Scheme-versus-Scheme Accuracy Comparison

Table 8.3: Accuracy on MNIST non-IID under a 50% malicious-client rate. Entries are percentages; baseline values are from [1].

Scheme	**Targeted** (T_{acc})	**Untargeted** (ACC)
Krum [8]	71.4	78.3
Auror [11]	32.5	49.9
PEFL [13]	46.4	57.4
Sybils / FoolsGold [15]	89.7	89.5
FL-Trust [12]	37.8	43.4
ShieldFL [16]	90.1	89.4
Auditor-based [1]	96.5	96.3
FIP-FL (this thesis)	**97.69**	88.82

FIP-FL gives the best targeted accuracy in the table while using weaker trust assumptions than the auditor-based baseline. Its untargeted result is competitive with FoolsGold and ShieldFL but below the auditor-based row; this gap is the clearest empirical target for future improvement. The important point for this thesis is that FIP-FL achieves the strongest targeted recovery without decrypted cluster statistics or an external auditor.

9 Conclusion and Future Work

9.1 Summary of the Thesis

This thesis argued that Byzantine-resilient privacy-preserving federated learning does not require an external auditor that decrypts cluster structure. FIP-FL replaces that trust anchor with a functional inner product between each encrypted client update and a trusted momentum-tracked reference direction. The resulting scalar score supports a two-stage geometric audit: a direction filter for label-flip style reversals and a magnitude window for under-scaled or over-scaled updates.

The framework combines Paillier encryption, homomorphic score computation, secure aggregation of accepted ciphertexts, and momentum-based reference updating. The theoretical analysis establishes scalar-gradient privacy, a geometric admissibility condition for adversaries, and monotone loss decrease under the stated L-smoothness assumptions. Empirically, FIP-FL detects every completed malicious-client configuration in the evaluation grid, restores MNIST non-IID target accuracy from roughly 7% undefended to 97.69% at a 50% malicious-client rate, and removes the auditor's $\mathcal{O}(d^3)$ covariance-inversion step while giving a 13× parallel speed-up in Phase II.

9.2 Limitations and Future Work

FIP-FL deliberately uses one reference direction per round. This keeps verification linear and efficient, but highly heterogeneous federations may contain several honest descent directions at once; a single reference can therefore under-admit useful weakly aligned clients. A multi-reference cone $\{\mathbf{R}_k^{(t)}\}_{k=1}^{K}$ is the natural extension, with safeguards against cone-overlap exploits.

The present evaluation also uses logistic-regression models with dimensions in the low thousands, a fixed client set, and one attack mode per run. Deep-network-scale deployment would require packed homomorphic encryption such as BFV or CKKS [5], while realistic deployments should study client churn, changing data distributions, adaptive adversaries, and mixed attacks such as label flipping combined with scaling.

Finally, the initial reference is computed from a small public bootstrap set. A stronger first-round procedure could form the reference from robust aggregate statistics rather than designated public data, preserving the no-individual-gradient-decryption rule while reducing bootstrap dependence. End-to-end deployment on laptop, mobile, and edge clients would then test the practical balance between encryption cost, communication cost, and model utility in the settings where PPFL is most needed.

Bibliography

[1] Abbas Yazdinejad, Ali Dehghantanha, Hadis Karimipour, Gautam Srivastava, and Reza M. Parizi. A robust privacy-preserving federated learning model against model poisoning attacks. *IEEE Transactions on Information Forensics and Security*, 19:6693–6708, 2024.

[2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 54, pages 1273–1282, 2017.

[3] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[4] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology – EUROCRYPT '99*, pages 223–238. Springer, 1999.

[5] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer, 2017.

[6] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2938–2948, 2020.

[7] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 634–643, 2019.

[8] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine-tolerant gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[9] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5650–5659, 2018.

[10] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 3521–3530, 2018.

[11] Shiqi Shen, Shruti Tople, and Prateek Saxena. Auror: Defending against poisoning attacks in collaborative deep learning systems. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, pages 508–519, 2016.

[12] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium (NDSS)*, 2021.

[13] Xiaoyuan Liu, Hongwei Li, Guowen Xu, Zongqi Chen, Xiaoming Huang, and Rongxing Lu. Privacy-enhanced federated learning against poisoning adversaries. *IEEE Transactions on Information Forensics and Security*, 16: 4574–4588, 2021.

[14] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pages 1175–1191, 2017.

[15] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in Sybil settings. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pages 301–316, 2020.

[16] Zhuoran Ma, Jianfeng Ma, Yinbin Miao, Yuan Li, and Robert H. Deng. ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security*, 17:1639–1654, 2022.

[17] Anthony Zhou and Amir Barati Farimani. Hamiltonian neural PDE solvers through functional approximation, 2025.